PROJET D’APPLICATION FULL STACK

SALES & DATA QUALITY PLATFORM

Plateforme collaborative d’analyse des ventes
et de contrôle de la qualité des données

RAPPORT DE PROJET

Conception, réalisation et validation d’un MVP

Équipe	Responsabilité fonctionnelle
Yassine	Fondation, authentification, autorisations et administration
Ahmed	Gestion des datasets et importation CSV
Taha	Exploration, analyses, qualité et comparaisons
Abdelali	Visualisation, dashboards et partage

Octobre 2026

Données de démonstration fictives · Version locale du projet

Résumé et repères de lecture

Du fichier CSV à une analyse enregistrée et partageable

La plateforme répond à un besoin simple : rendre un fichier de ventes exploitable sans multiplier les outils. Elle associe un import traçable, une exploration filtrée, des indicateurs financiers, un contrôle de qualité et des dashboards partagés entre utilisateurs inscrits. Le parcours est protégé par des sessions gérées côté serveur et des permissions vérifiées sur chaque ressource.

Le MVP utilise React et TypeScript pour le client, Express pour l'API, PostgreSQL pour les données et Prisma pour le modèle et les migrations. La séparation en quatre modules permet de conserver une répartition claire : Yassine pour le socle et l'administration, Ahmed pour l'ingestion, Taha pour les analyses et Abdelali pour les dashboards.

Démonstration fictive	Résultat vérifié
Import du fichier de référence	10 lignes sources : 8 acceptées, 2 rejetées
Indicateurs métier	141,75 GBP de CA net ; 6 commandes ; panier moyen 23,63 GBP
Qualité et validation	Score 90/100 ; 12 tests automatisés réussis

Table des matières

1. Contexte et objectifs 3

2. Besoins et parcours utilisateur 4

3. Organisation et responsabilités 5

4. Architecture et choix techniques 6

5. Modèle de données partagé 7

6. Contrats API et conventions 8

7. Authentification et profil — Yassine 9

8. Administration et permissions — Yassine 10

9. Import CSV — Ahmed 11

10. Fiabilité du traitement — Ahmed 12

11. Exploration et filtres — Taha 13

12. Indicateurs métier — Taha 14

13. Qualité des données — Taha 15

14. Valeurs atypiques et comparaison — Taha 16

15. Dashboards et graphiques — Abdelali 17

16. Partage et contrôle des accès — Abdelali 18

17. Vérification et résultats 19

18. Installation, limites et conclusion 20

1 Contexte et objectifs

Transformer des données de ventes hétérogènes en informations compréhensibles

1.1 Problématique

Un fichier CSV peut contenir des ventes utiles tout en présentant des dates ambiguës, des champs manquants, des nombres invalides ou des retours. Une analyse réalisée directement dans un tableur risque de masquer ces difficultés. Le nombre de lignes ne correspond pas forcément au nombre de commandes, et additionner les montants sans traiter les annulations produit des indicateurs trompeurs.

Le projet vise donc à relier deux dimensions : la lecture commerciale des ventes et la qualité des données qui alimentent cette lecture. L'utilisateur doit pouvoir identifier les lignes rejetées, comprendre les règles de calcul et retrouver les filtres utilisés dans un dashboard enregistré.

1.2 Objectifs opérationnels

Objectif	Résultat attendu
Importer avec traçabilité	Conserver le fichier, les valeurs originales et les numéros de lignes.
Analyser de façon cohérente	Appliquer les mêmes définitions de dates, devises, commandes et retours.
Visualiser simplement	Présenter les KPI et trois graphiques prédéfinis avec des filtres communs.
Collaborer sans exposer les données	Partager une vue en lecture seule sans ouvrir le dataset brut.
Administrer le service	Gérer les comptes et suivre les métadonnées et activités.

1.3 Périmètre retenu

La réalisation couvre un MVP local utilisant des données fictives. Elle comprend l'authentification, l'import asynchrone, les analyses, les dashboards et le partage nominatif. Elle ne revendique ni un déploiement de production, ni une capacité mesurée sur de gros volumes. Les fonctions avancées restent explicables : score de qualité documenté, méthode IQR et comparaison de périodes ou de datasets.

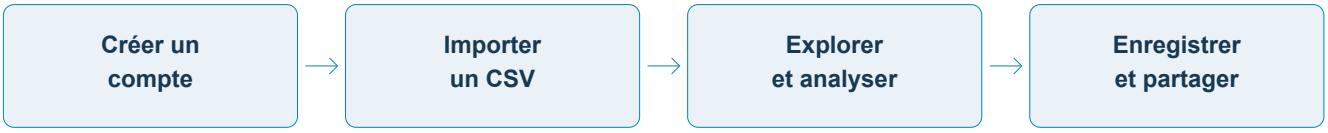
Question directrice : comment proposer, dans une seule application, un parcours CSV → exploration → analyse → visualisation → partage, tout en maintenant des calculs cohérents et des données privées ?

2 Besoins et parcours utilisateur

Un parcours complet avec des états visibles à chaque étape

2.1 Acteurs du système

Le visiteur consulte la présentation et crée un compte ou se connecte. Le membre connecté possède ses datasets et ses dashboards ; il peut aussi recevoir un accès à un dashboard partagé. L'administrateur dispose d'écrans de supervision et de gestion des utilisateurs. Le statut d'administrateur ne lui accorde pas automatiquement la lecture des ventes privées d'un membre.



2.2 Scénario principal

Après sa connexion, l'utilisateur crée un dataset en renseignant son titre, sa devise et ses paramètres de date. Il dépose le CSV et consulte l'état du traitement. Une fois l'import terminé, il examine le nombre de lignes acceptées et rejetées. Les lignes acceptées alimentent les analyses, tandis que le rapport des rejets explique les corrections à apporter à la source.

L'utilisateur choisit ensuite une période, un pays ou un produit. Les indicateurs et les visualisations reflètent ces filtres. Il enregistre un dashboard pour retrouver sa configuration et peut inviter un utilisateur inscrit à consulter cette vue. Il conserve la possibilité de révoquer ce partage.

2.3 Besoins non fonctionnels

Exigence	Réponse du MVP
Confidentialité	Session obligatoire, contrôle du propriétaire, partage limité à une vue.
Fiabilité	Transactions, états persistés, détection d'un fichier identique et reprise.
Compréhension	Erreurs lisibles, chargements, listes vides et limites expliquées.
Maintenabilité	TypeScript, modules métier, contrats communs et migrations versionnées.
Reproductibilité	Seed fictif, CSV de référence et résultats calculés manuellement.

Les écrans distinguent une absence de données, un chargement en cours, une validation refusée et une erreur technique. Ces états évitent de présenter un graphique vide comme une absence certaine de ventes alors que le traitement n'est pas encore terminé.

3 Organisation et responsabilités

Quatre domaines fonctionnels reliés par un contrat commun

Le dépôt est organisé autour des responsabilités prévues pour l'équipe. Cette répartition indique les parties à présenter et à maintenir ; elle ne constitue pas une attestation des contributions individuelles dans l'historique Git. Les fichiers partagés restent coordonnés par Yassine pour éviter des contrats incompatibles entre modules.

Membre	Périmètre et livrables
Yassine	Initialisation client/serveur, schéma commun, sessions, permissions, profil, gestion des utilisateurs et supervision.
Ahmed	Métadonnées des datasets, upload CSV, validation, normalisation, jobs, rapports de rejet et suppression cohérente.
Taha	Tableau paginé, filtres, indicateurs, qualité, valeurs atypiques et comparaison. Publie les calculs réutilisés par les dashboards.
Abdelali	KPI et graphiques, sélection du dataset, configuration sauvegardée, consultation et partage en lecture seule.

3.1 Organisation du code

```
server/src/modules/{auth,admin,datasets,analytics,dashboards}
client/src/features/{auth,admin,datasets,analytics,dashboards}
docs/                  contrats et documents de coordination
server/prisma/         schema et migrations de la base
```

3.2 Coordination et intégration

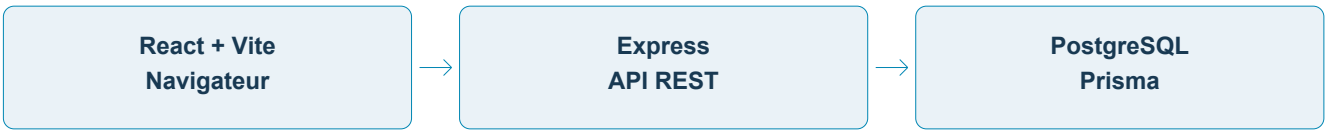
AGENTS.md définit les règles communes. Le contrat OpenAPI décrit les requêtes et réponses ; le contrat de données fixe les types et les règles de vente. Toute évolution du schéma nécessite une migration supplémentaire, sans réécrire une migration déjà appliquée. Une modification publiée de contrat doit être documentée et coordonnée.

Les branches prévues sont feature/auth-admin, feature/datasets, feature/analytics et feature/dashboards. Le socle intégré est identifié par foundation-v1.1. La version complète vérifiée se trouve sur feature/auth-admin ; ce rapport ne suppose pas une fusion de cette branche dans main.

Dépendance principale : Ahmed fournit les ventes normalisées ; Taha calcule les agrégats ; Abdelali les affiche. Yassine fournit à tous la session, les droits et les conventions partagées.

4 Architecture et choix techniques

Une application en trois couches dans un dépôt unique



4.1 Client web

Le dossier client contient une application React en TypeScript construite avec Vite. React Router gère les pages publiques et les routes protégées. Un layout commun rassemble la navigation, l'utilisateur connecté et les états généraux. Les domaines métier sont regroupés dans features afin que chaque partie reste identifiable.

Recharts 3 est la bibliothèque commune de graphiques. Les écrans utilisent les données renvoyées par l'API ; ils ne recréent pas une seconde logique financière dans le navigateur. Les conversions en nombres JavaScript sont réservées à l'affichage des graphiques.

4.2 Serveur et persistance

Le serveur Express en TypeScript expose des endpoints sous /api. Les middlewares partagés authentifient la session, valident les entrées et contrôlent les droits. Les modules métier délèguent la persistance à Prisma et PostgreSQL. Les agrégations financières utilisent des décimaux et des requêtes paramétrées.

Un traitement simple, exécuté dans le processus serveur, récupère périodiquement les imports à traiter. Les fichiers et les états étant persistés en base, un redémarrage ne dépend pas de la mémoire du processus pour retrouver les jobs. Ce choix limite la complexité du MVP, mais n'équivaut pas à une file de tâches distribuée.

Technologie	Rôle dans la réalisation
React / Vite / TypeScript	Interface, compilation et typage.
Node.js / Express	API REST et orchestration des traitements.
PostgreSQL 16 / Prisma	Contraintes relationnelles, transactions et migrations.
npm workspaces / Docker Compose	Installation commune et base locale reproductible.
Recharts 3	Courbe temporelle, barres produits et répartition par pays.

Les échanges suivent une séparation claire : le client demande une opération, le serveur en vérifie la validité et les permissions, puis la base garantit la cohérence des relations et des écritures.

5 Modèle de données partagé

Préserver la source et distinguer données brutes, données propres et vues

Le schéma commun comprend dix tables. Les identifiants métier de vente restent des chaînes, tandis que les identifiants internes sont des UUID. La séparation entre fichier source, lignes brutes et ventes normalisées permet d'expliquer les résultats et de reprendre un traitement sans perdre l'origine des données.

Table	Champs ou rôle principal
User / Session	Compte, rôle, activation ; empreinte du token et expiration de session.
Dataset	Propriétaire, titre, description, catégorie, source, devise, format de date et fuseau.
ImportJob / ImportSource	État, empreinte SHA-256, compteurs, reprises ; contenu original binaire.
RawRecord	Valeurs JSON originales, ligne source logique et erreurs de validation.
SalesRecord	Facture, produit, description, quantité, date, prix, client facultatif et pays.
Dashboard / DashboardShare	Propriétaire, dataset, configuration ; utilisateurs autorisés à lire la vue.
Activity	Action administrative ou métier et acteur associé, lorsque disponible.

5.1 Relations et contraintes

Un utilisateur possède plusieurs datasets et dashboards. Un dataset regroupe ses imports, ses lignes et ses dashboards. Chaque vente est rattachée à un dataset et à un import cohérents grâce aux contraintes relationnelles. L'unicité de l'empreinte par dataset empêche le double import du même fichier. Les lignes identiques dans un fichier ne sont pas supprimées automatiquement.

Le propriétaire d'un dashboard doit être celui du dataset associé. Une ligne de partage relie le dashboard à un utilisateur inscrit. La suppression autorisée d'un dataset entraîne celle de ses imports, sources, lignes, dashboards et partages dépendants.

5.2 Types et précision

Les quantités et prix utilisent des colonnes Decimal(18,4). Le client facultatif reste nullable. Les dates sont stockées en UTC. Le dataset déclare sa devise : aucun taux de change implicite n'est appliqué. Les montants calculés quittent l'API sous forme de chaînes décimales pour éviter une perte de précision pendant le transport JSON.

6 Contrats API et conventions

Des formats identiques pour tous les modules

Le contrat OpenAPI de docs/api-contract.yaml décrit 33 chemins dans sa version 1.1.0. Il regroupe les opérations de session, d'administration, de datasets, d'analyses et de dashboards. Les exemples de réponses aident les développeurs à avancer sans inventer une structure différente pour chaque écran.

Groupe	Fonctions principales
/api/auth/*	Inscription, connexion, déconnexion, utilisateur courant et profil.
/api/admin/*	Utilisateurs, résumé, datasets et activités.
/api/datasets/*	Création, liste, détail, imports, rejets et suppression.
/api/datasets/:id/analytics/*	Exploration, KPI, agrégats, qualité, IQR et comparaison.
/api/dashboards/*	CRUD, agrégats de la vue et partage nominatif.

6.1 Erreurs et pagination

```
{ "error": { "code": "UNAUTHORIZED",
  "message": "Authentication required", "requestId": "..." } }

{ "data": [], "meta": { "page": 1, "pageSize": 20,
  "total": 0, "totalPages": 0 } }
```

Le format d'erreur contient un code exploitable, un message et un identifiant de requête ; des détails de validation peuvent être ajoutés. La pagination commence à 1, utilise 20 éléments par défaut et limite la taille à 100. Les paramètres inattendus ou invalides sont rejetés selon les règles de chaque endpoint.

6.2 Filtres et périmètre d'accès

Les filtres de date utilisent une borne from inclusive et une borne to exclusive, avec un horodatage RFC3339 comportant un décalage. Les filtres de pays et de produit se combinent avec la période par une condition ET. La recherche est traitée comme du texte et les colonnes de tri sont choisies dans une liste autorisée.

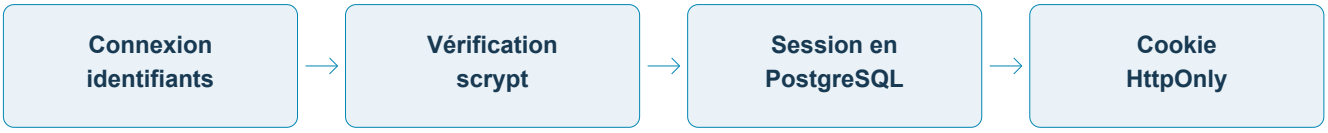
Un accès direct aux analyses exige la propriété du dataset. Le destinataire d'un partage utilise exclusivement l'endpoint de données du dashboard ; il ne reçoit pas de droit général sur les enregistrements du dataset. Un endpoint de santé GET /api/health permet de contrôler que le serveur répond.

7 Authentification et profil

Partie Yassine — gérer la session et protéger les opérations sensibles

7.1 Cycle de connexion

L'inscription crée un utilisateur ordinaire. Le serveur refuse les champs permettant de choisir son rôle ou son propriétaire. Les mots de passe ne sont pas stockés en clair : ils sont dérivés avec scrypt et un sel propre à chaque compte. Lors de la connexion, le serveur vérifie le mot de passe et l'activation du compte avant de créer une session.



Le cookie contient un token opaque aléatoire de 256 bits ; la base conserve uniquement son empreinte SHA-256. La durée par défaut est de 24 heures. Le cookie HttpOnly évite une lecture par le JavaScript de l'application. Les options SameSite et les réglages Secure de production complètent ce dispositif. Aucun token de session n'est stocké dans localStorage.

7.2 Middleware et routes protégées

Les routes frontend attendent la résolution de l'utilisateur courant avant d'afficher les pages privées. Le contrôle décisif reste côté serveur : le middleware authentifie la session, puis les helpers vérifient le rôle ou la propriété de la ressource. Les mutations exigent une origine autorisée et un en-tête applicatif afin de réduire le risque de requêtes intersites non souhaitées.

7.3 Gestion du profil

Le membre peut modifier son nom et son email. Un changement d'email demande le mot de passe courant et révoque les autres sessions. Le changement de mot de passe exige aussi l'ancien mot de passe et révoque les sessions. La déconnexion invalide la session serveur et supprime le cookie, au lieu de se limiter à masquer l'utilisateur dans l'interface.

Vérifications réalisées : connexion valide, mauvais mot de passe, absence de session, déconnexion, expiration, refus d'injection de rôle et révocation après une opération sensible. Les tests couvrent également des opérations concurrentes de connexion et de changement de mot de passe.

8 Administration et permissions

Partie Yassine — superviser sans contourner la confidentialité des données

L'écran d'administration regroupe un résumé global, les utilisateurs, les métadonnées des datasets et les activités. Les listes sont paginées. L'administrateur peut modifier le rôle ou l'activation d'un compte ; les endpoints correspondants sont protégés par un middleware spécifique et restent interdits à un utilisateur ordinaire.

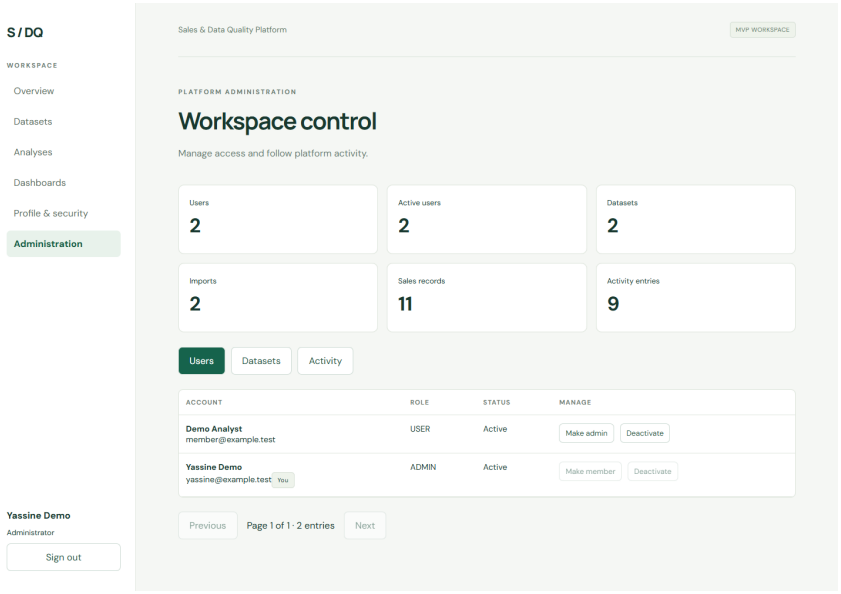


Figure 1 — Écran réel de supervision : indicateurs, comptes et actions d'administration.

8.1 Garde-fous sur les changements de rôle

Un administrateur ne peut pas se désactiver ou se retirer lui-même ses droits par cette interface. La protection du dernier administrateur et les contrôles transactionnels évitent de laisser le système sans responsable. Une modification sensible révoque les sessions du compte concerné ; la vérification en base empêche une ancienne session de continuer à agir avec un rôle périmé.

8.2 Journalisation et séparation des droits

Les activités conservent les actions utiles à la supervision. Les résumés portent sur les comptes et les métadonnées, pas sur un accès implicite aux lignes de vente privées. Les contrôles de propriété restent applicables à l'administrateur lorsqu'il utilise les endpoints de datasets ou d'analyses.

Acteur	Accès autorisé
Membre	Son profil, ses datasets, ses dashboards et les vues partagées avec lui.
Administrateur	Gestion des comptes et supervision ; pas de lecture automatique des ventes d'autrui.
Session révoquée	Aucune action protégée sans nouvelle authentification.

9 Importation CSV et validation

Partie Ahmed — créer le dataset et conserver la traçabilité des lignes

La création d'un dataset précise le titre, la description, la catégorie, la source, la devise, le format de date et le fuseau horaire. Les opérations de consultation, modification, import et suppression vérifient la propriété. Un autre utilisateur ne peut pas accéder à un dataset en devinant son identifiant.

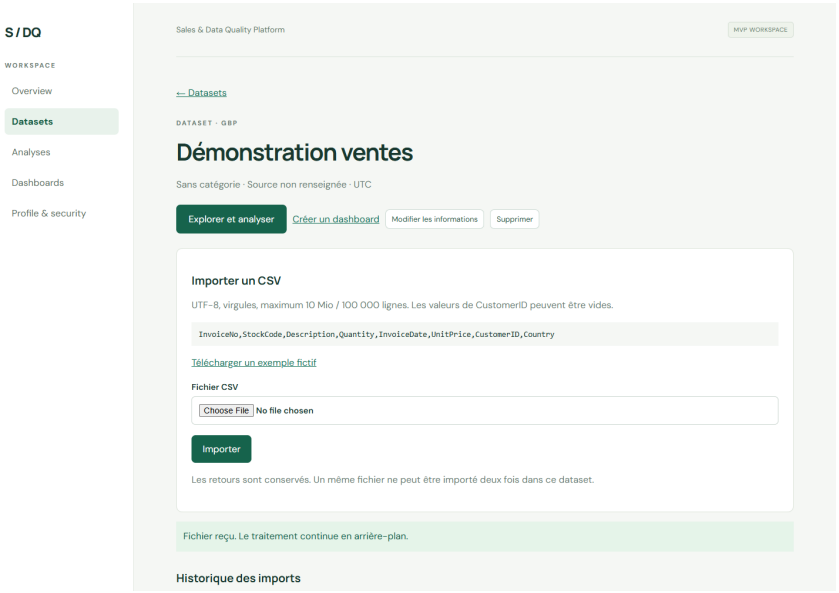


Figure 2 — Gestion du dataset et dépôt d'un CSV dans l'interface réelle.

9.1 Colonnes acceptées

Le fichier UTF-8 peut comporter un BOM, des champs entre guillemets et des virgules dans les valeurs correctement échappées. Les en-têtes convenus sont InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID et Country. Le client est facultatif ; les autres champs requis sont contrôlés. Les codes et numéros restent des chaînes pour préserver les zéros initiaux.

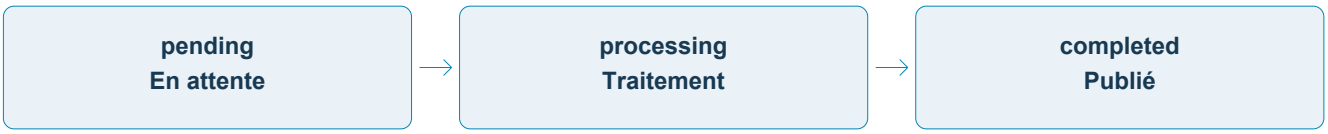
9.2 Contrôles et rapport de rejet

L'upload est limité à 10 Mio, à 100 000 lignes logiques et à 64 Kio UTF-8 par champ. Les nombres, prix non négatifs, champs obligatoires et dates sont validés. Une date impossible est rejetée. Les ambiguïtés de fuseau et de changement d'heure sont traitées explicitement plutôt que corrigées silencieusement.

Les valeurs d'origine et le numéro logique de ligne sont conservés dans les enregistrements bruts. Le rapport associe chaque rejet à une raison lisible. Un CSV structurellement mal formé peut faire échouer le job entier ; une ligne métier invalide est comptée parmi les rejets d'un import terminé.

10 Fiabilité du traitement des imports

Partie Ahmed — états persistés, reprise et publication atomique



10.1 Traitement hors de la requête HTTP

Le dépôt d'un fichier crée un job et stocke sa source en base. La requête n'attend pas la validation de toutes les lignes. Un worker simple du serveur recherche périodiquement les jobs à traiter. Le job enregistre son état et ses compteurs ; en cas d'échec, il passe à failed et conserve les informations permettant une reprise.

Un verrou consultatif PostgreSQL empêche deux traitements simultanés du même job. L'écriture des lignes brutes, des ventes propres et des compteurs définitifs est transactionnelle. Les analyses ne lisent que les imports terminés. Un échec ne rend donc pas visibles des ventes partiellement importées.

10.2 Réimport et reprise après incident

L'empreinte SHA-256 est calculée sur les octets du fichier. Le couple dataset–empreinte est unique : un deuxième envoi du même fichier dans ce dataset est refusé. Cette règle concerne le fichier, pas les transactions individuelles ; deux lignes identiques peuvent être légitimes et restent conservées.

Le contenu original reste disponible dans ImportSource. Une relance réutilise le job et incrémente son compteur de reprise. Un job interrompu pendant processing peut être repris après redémarrage. Le verrou et la transaction évitent une double publication. La suppression du dataset nettoie également les sources et les objets dépendants.

Cas contrôlé	Comportement attendu
Quantité « abc »	Rejet de la ligne avec numéro source et motif.
Fichier déjà importé	Refus de créer un import identique dans le même dataset.
CSV mal formé / job échoué	Aucune vente partiellement publiée ; reprise documentée.
Requête d'un autre membre	Accès refusé avant toute lecture ou modification.

Exemple fourni : fixtures/ventes-demo.csv contient 10 lignes de données. Les 8 premières sont acceptées ; les lignes sources 10 et 11 sont rejetées, l'en-tête occupant la première ligne. Les motifs sont une quantité invalide et une date impossible.

11 Exploration et filtres

Partie Taha — examiner les ventes sans charger tout le dataset dans le navigateur

11.1 Tableau de données

L'explorateur présente les enregistrements acceptés et publiés. La pagination, la recherche et le tri sont exécutés côté serveur. La page courante contient uniquement les résultats nécessaires à l'écran ; les métadonnées indiquent le total et le nombre de pages. Les champs de tri sont explicitement autorisés afin d'éviter une colonne arbitraire dans une requête.

L'utilisateur peut rechercher une facture ou un produit puis combiner une période, un pays et un produit. Les mêmes conventions s'appliquent au tableau et aux agrégats. Une période sans vente renvoie une liste vide et des indicateurs définis, plutôt qu'une erreur technique.

11.2 Dates et fuseaux horaires

À l'import, un format ISO 8601 explicite ou le format documenté DMY_HM est interprété avec le fuseau du dataset. Les dates sont ensuite enregistrées en UTC. Les filtres saisis comme journées dans l'interface sont explicitement convertis en bornes UTC : début inclus, lendemain du dernier jour exclu. Cette convention évite un doublon entre deux périodes consécutives.

Les regroupements temporels journaliers, hebdomadaires ou mensuels utilisent le fuseau du dataset. Il faut donc distinguer la sélection d'un intervalle UTC et la façon d'étiqueter les périodes de regroupement. Le contrat rend cette différence visible pour éviter des désaccords entre écrans.

Contrôle	Règle partagée
Pagination	page ≥ 1 ; pageSize par défaut 20, maximum 100.
Recherche	Texte littéral insensible à la casse ; paramètres validés.
Tri	Champ choisi dans une liste autorisée.
Période	from inclus ; to exclu ; décalage horaire explicite.
Pays et produit	Combinaison avec les autres filtres par une condition ET.
Sécurité	Propriété du dataset contrôlée avant les requêtes.

Exemple de borne : pour deux périodes consécutives séparées au 12 janvier à 00:00 UTC, une vente exactement à cet instant appartient uniquement à la deuxième période. Les tests couvrent les dates invalides et les intervalles inversés.

12 Indicateurs métier et calculs décimaux

Partie Taha — compter les commandes et intégrer correctement les retours

12.1 Définitions communes

Le chiffre d'affaires net est la somme des quantités signées multipliées par les prix unitaires. Le chiffre d'affaires brut additionne les montants positifs ; les retours sont exprimés comme la valeur absolue des montants négatifs. Une facture commençant par C signale une annulation et sa quantité est normalisée en négatif. Une quantité déjà négative reste un retour valide.

CA net = somme(quantité signée × prix unitaire)
Commandes = nombre de factures distinctes ayant une vente positive hors annulation
Panier moyen = CA net / commandes ; valeur nulle si aucune commande

Les calculs utilisent PostgreSQL numeric et des décimaux Prisma de précision élevée. L'arrondi monétaire à deux décimales intervient après agrégation, selon la règle half-up. L'API renvoie les montants sous forme de chaînes. Les datasets de devises différentes ne sont pas additionnés comme s'ils étaient directement comparables.

12.2 Vérification manuelle du fichier fictif

Élément	Calcul ou résultat attendu
Facture INV001	4 × 5,25 + 2 × 8,50 = 38,00 GBP ; une seule commande.
Autres ventes positives	6,00 + 1,00 + 1,00 + 1,00 + 100,00 = 109,00 GBP.
Annulation C003	−1 × 5,25 = −5,25 GBP.
Résultat net	147,00 − 5,25 = 141,75 GBP.
Commandes et panier	6 factures positives distinctes ; 141,75 / 6 = 23,63 GBP.
Répartition géographique	France : 109,00 GBP ; Royaume-Uni : 32,75 GBP.

Deux lignes appartenant à INV001 constituent donc une commande, pas deux. Un résultat vide produit un CA de 0,00, zéro commande et un panier moyen nul. Le client distingue une valeur nulle d'un panier de zéro ; cette distinction reste utile dans une comparaison de périodes.

13 Rapport de qualité des données

Partie Taha — expliquer le score au lieu de masquer les défauts

Le rapport de qualité sépare les lignes sources rejetées des ventes propres acceptées. Il présente les valeurs manquantes, la validité des sources et les candidats doublons. Le score porte sur les imports terminés du dataset ; les filtres commerciaux n'en changent pas artificiellement la qualité globale, même si un compteur décrit les enregistrements filtrés.

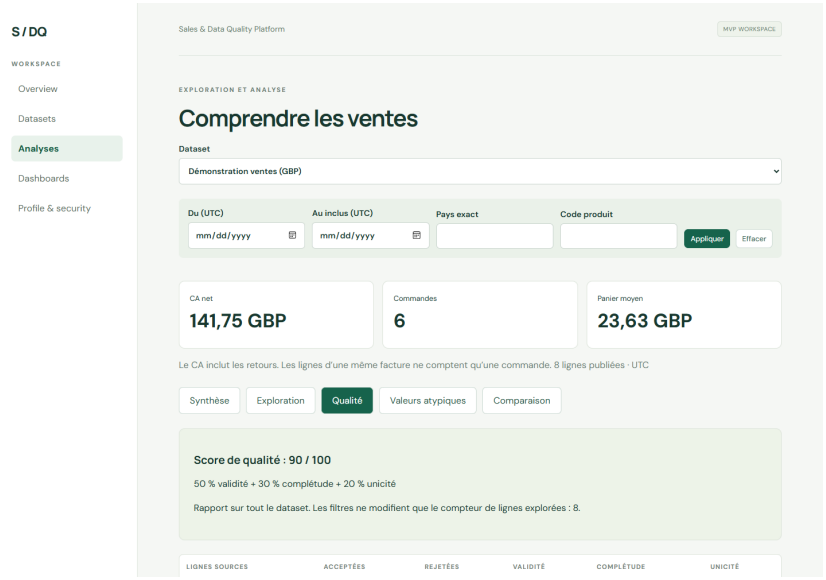


Figure 3 — Rapport réel de qualité : score et indicateurs du dataset fictif.

13.1 Formule publiée

Score = 0,50 × validité + 0,30 × complétude + 0,20 × unicité
Validité = 100 × acceptées / sources
Complétude : présence des sept champs obligatoires
Unicité = 100 × (1 − candidats doublons / acceptées)

CustomerID est facultatif : ses valeurs manquantes sont rapportées, mais ne pénalisent pas la complétude des sept champs obligatoires. Un candidat doublon correspond à une répétition exacte du tuple normalisé des huit champs ; pour un groupe de taille n, n − 1 lignes sont candidates. Cette définition n'affirme pas qu'une vente est dupliquée par erreur.

13.2 Application à l'exemple

Les 8 lignes acceptées sur 10 donnent une validité de 80 %. Tous les champs obligatoires acceptés sont présents et aucun tuple normalisé n'est répété : complétude et unicité valent 100 %. Le score vaut donc 0,50 × 80 + 0,30 × 100 + 0,20 × 100 = 90. Un client absent est signalé. Sans source exploitable, le score vaut zéro selon la convention du MVP.

14 Valeurs atypiques et comparaison

Partie Taha — signaler les anomalies et comparer des groupes explicites

14.1 Méthode IQR

La détection s'applique à la quantité et au prix unitaire. Après tri, les quartiles Q1 et Q3 sont obtenus par interpolation linéaire et IQR = Q3 − Q1. Une valeur est signalée si elle est strictement inférieure à Q1 − 1,5 × IQR ou supérieure à Q3 + 1,5 × IQR. Les seuils et la méthode sont exposés pour rendre le résultat interprétable.

Les ventes de quantité non négative et les retours négatifs sont examinés séparément. Cette séparation évite de classer un retour comme anormal uniquement parce que son signe diffère de celui des ventes. Les groupes de moins de quatre observations ne produisent pas de signalement. Avec un IQR nul, seules les valeurs différentes des bornes identiques sont signalées.

Dans le jeu fictif, la quantité 100 mérite une vérification. Un prix plus élevé peut aussi être signalé selon les quartiles du groupe. Le résultat n'est pas une preuve d'erreur : une vente importante ou un produit coûteux peut être légitime. Aucune ligne n'est supprimée automatiquement.

14.2 Comparer deux périodes ou deux datasets

La comparaison présente pour chaque groupe son libellé, son CA net, ses commandes et son panier moyen. Les deux datasets doivent être accessibles à l'utilisateur et utiliser la même devise. La différence absolue est actuel − référence. La variation relative est 100 × différence / valeur absolue de la référence.

Situation	Interprétation
Référence 100 ; actuel 125	Écart +25 ; variation +25 %.
Référence 0 ; actuel 10	Écart +10 ; pourcentage nul/non calculable, pas une hausse infinie.
Aucune commande	Panier moyen nul ; variation du panier non calculable.
Devises distinctes	Comparaison refusée sans conversion explicite.
Dataset non autorisé	Accès refusé, y compris si l'autre dataset est accessible.

Portée : IQR est une méthode descriptive et explicable. La comparaison mesure un écart observé ; elle ne prouve pas sa cause et ne réalise aucune prévision.

15 Dashboards et visualisation

Partie Abdelali — enregistrer une vue cohérente des analyses

Un dashboard associe un propriétaire, un dataset et une configuration validée. L'utilisateur choisit les widgets disponibles, enregistre ses filtres, renomme la vue et peut la supprimer. La configuration sauvegardée permet de retrouver les mêmes choix après un rechargement ou une nouvelle connexion.

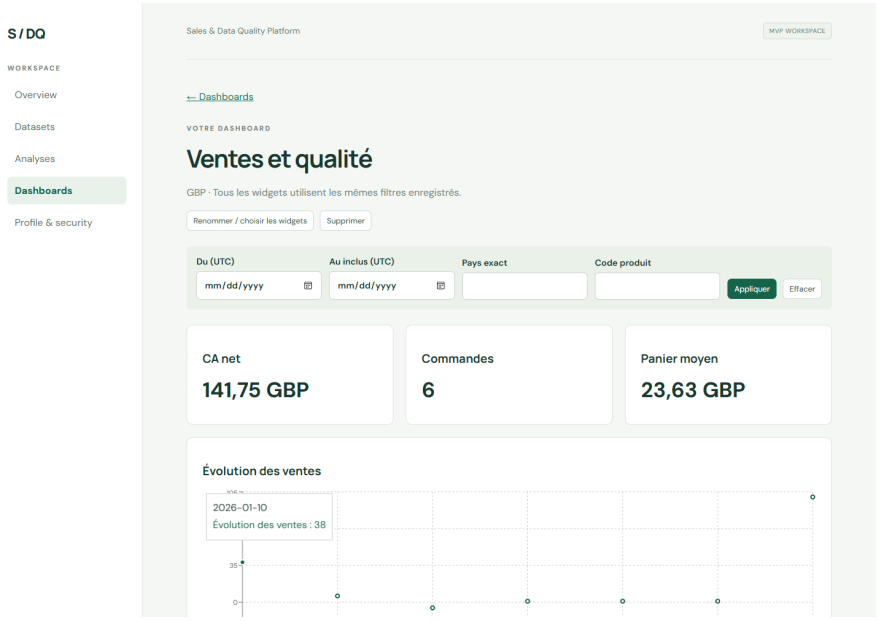


Figure 4 — Dashboard réel : KPI financiers et visualisations du dataset de démonstration.

15.1 Widgets prédéfinis

Widget	Information présentée
Trois KPI	CA net, nombre de commandes et panier moyen.
Courbe temporelle	Évolution du chiffre d'affaires sur les périodes.
Barres par produit	Produits classés selon le montant calculé.
Répartition par pays	Groupes de montant positif dans le diagramme circulaire.

15.2 Réutilisation des analyses

Les widgets utilisent les services de calcul de Taha. Tous partagent la même configuration de filtres et la même devise. Les KPI, barres et courbes conservent les montants signés des retours. Le diagramme circulaire représente seulement les groupes positifs, avec une mention visible de cette limite.

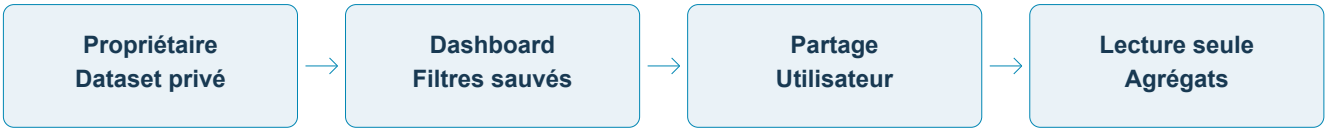
La configuration est versionnée et contrôlée côté serveur : types de widgets connus, taille bornée et grille sans chevauchement. Le MVP limite le nombre de widgets à huit. Cette validation évite qu'un document JSON arbitraire enregistré en base impose un affichage incohérent ou une requête non autorisée.

16 Partage et contrôle des accès

Partie Abdelali, avec le socle Yassine — ouvrir une vue sans ouvrir le dataset

16.1 Partage nominatif en lecture seule

Le propriétaire ajoute un utilisateur inscrit et actif, identifié par son email ou son UUID. Le destinataire peut consulter le dashboard partagé et ses agrégats autorisés. Il ne peut ni modifier la configuration, ni gérer les partages, ni supprimer la vue. Aucun lien public anonyme n'est nécessaire pour ce scénario.



Les données de la vue passent par un endpoint spécifique. Celui-ci contrôle le propriétaire ou l'existence du partage puis réutilise les agrégats du service analytics. Il n'accepte pas de filtres arbitraires en requête qui permettraient d'étendre le périmètre enregistré. Le destinataire ne peut donc pas transformer un partage limité en un accès général à toutes les ventes.

16.2 Matrice d'autorisation

Opération	Propriétaire	Destinataire
Lire le dashboard et ses agrégats	Oui	Oui, partage actif
Renommer, filtrer, configurer	Oui	Non
Supprimer / gérer le partage	Oui	Non
Lire les lignes brutes du dataset	Oui	Non via le partage
Après révocation	Oui	Non

16.3 Révocation et ressources liées

Le retrait du partage supprime l'autorisation dès la requête suivante ; une session encore ouverte ne suffit pas à conserver le droit. La suppression du dataset entraîne celle des dashboards associés et de leurs partages. Les ressources inaccessibles sont protégées par les réponses convenues, sans renvoyer leur contenu privé.

Scénario vérifié : un second utilisateur ouvre une vue partagée, ne peut pas la modifier ni lire le dataset, puis perd l'accès après révocation. Le contrôle est appliqué par le serveur et ne dépend pas seulement des boutons affichés.

17 Vérification et résultats

Des résultats reproductibles sur un jeu fictif, sans extrapolation de performance

17.1 Stratégie de vérification

La validation combine compilation, migrations, tests automatisés et parcours navigateur. Les tests créent des comptes isolés et utilisent une base de développement. Le CSV de référence fournit un résultat calculable à la main, ce qui permet de comparer les règles annoncées aux réponses réelles de l'API.

Domaine vérifié	Contrôles et résultat
Socle et sécurité	Login/logout, session absente ou expirée, rôle injecté, accès admin refusé ; réussis.
Profil et administration	Email/mot de passe, révocation, changements de rôles et courses concurrentes ; réussis.
Importation	Lignes invalides, fichier identique, CSV mal formé, reprise, absence de publication partielle ; réussis.
Analyses	Factures multiligne, retours, client absent, filtres, périodes vides, comparaison et IQR ; réussis.
Partage et suppression	Propriétaire, lecteur, révocation, absence d'accès au dataset et cascades ; réussis.
Compilation et navigateur	Builds client/serveur et scénario complet ; réussis. Vue mobile vérifiée.

17.2 Bilan observé

Les 12 tests automatisés de la version complète passent. La base a été générée, migrée et alimentée avec le seed fictif. Le parcours navigateur couvre l'import, les rejets, les analyses, le dashboard, le partage, la révocation et l'administration. La vérification mobile n'a pas montré de débordement horizontal sur le parcours contrôlé.

Les valeurs attendues sont retrouvées : 8 lignes acceptées, 2 rejetées, 141,75 GBP nets, 6 commandes, 23,63 GBP de panier moyen et 90/100 de qualité. Ces résultats démontrent la cohérence du scénario fourni ; ils ne constituent pas une mesure de débit, de disponibilité ou de sécurité exhaustive en production.

Version de référence : branche feature/auth-admin ; commit applicatif vérifié 10cd2ae3439ed535a03eb9be3925f7e2b3b361ad. Les documents détaillés sont produits à partir de cette base. La compilation signale un bundle de graphiques volumineux, sans bloquer le build.

18 Installation, limites et conclusion

Reproduire la démonstration et identifier les prochaines étapes

18.1 Démarrage local

Prérequis : Node.js 22.12 ou supérieur, npm, Docker Compose et PostgreSQL 16. Depuis la racine du dépôt, copier la configuration d'exemple puis installer, migrer et démarrer :

```
cp .env.example .env
npm ci
docker compose up -d --wait db
npm run db:generate
npm run db:migrate
npm run db:seed
npm run dev
```

L'interface est disponible sur http://localhost:5173 et l'API sur http://localhost:3001. Si le port de base est occupé, modifier ensemble DB_PORT et DATABASE_URL. Contrôles : npm run build puis npm test. Comptes fictifs locaux : yassine@example.test / DevAdmin123! et member@example.test / DevMember123!.

18.2 Limites et améliorations

Le MVP vise de petits volumes : les sources sont conservées en base et le worker reste dans le processus API. Le calcul IQR charge les colonnes numériques sélectionnées côté serveur. Aucun benchmark ne permet de promettre une montée en charge. La récupération de mot de passe, la vérification d'email, le MFA et une limitation distribuée des tentatives restent à ajouter avant une utilisation plus exposée.

Les évolutions prioritaires sont une file de tâches dédiée, un stockage de fichiers adapté, une supervision durable, des tests de charge et un déploiement de production. Les alertes IQR doivent rester des aides à la revue. Le partage reste nominatif et ne crée pas de publication publique des données.

18.3 Conclusion

Le parcours demandé fonctionne de bout en bout : un CSV est validé, normalisé, analysé puis représenté dans un dashboard sauvegardé et partagé. L'apport principal est la cohérence entre ingestion, métriques et permissions. Les responsabilités des quatre membres restent identifiées et les contrats communs rendent le projet plus facile à reprendre.

Sources et matériaux du projet

[1] Cahier des charges : CdC Application Full Stack.pdf, fourni par l'utilisateur.
[2] Dépôt : github.com/yassinechaoui333/dev_web ; README.md et docs/architecture.md.
[3] Contrats : docs/api-contract.yaml, docs/data-contract.md et docs/analytics-conventions.md.
[4] Vérification : server/test/, fixtures/ventes-demo.csv, fixtures/README.md et docs/integration-summary.md.
[5] Modèle de structure : Rapport_P4_Le_remplisseur_de_cluster_INPT_G4.pdf. Contenu métier adapté au présent projet ; captures issues de l'application locale.